

API Documentation

REST API conventions, endpoint reference, request/response formats, and status codes

Document status: Active

Version: 1.0

Last updated: 2026-09-26

1. API Conventions

FleetFlow services expose HTTP APIs using JSON.

General principles:

- Use nouns for resource paths.
- Use standard HTTP methods.
- Validate request bodies, query parameters, and route parameters.
- Return meaningful HTTP status codes.
- Keep authentication and authorization server-side.
- Avoid exposing secrets or sensitive implementation details.

2. Development Services

Service	Development Port	Purpose
Web	3000	Next.js frontend
Auth Service	4001	Authentication/users
Vehicle Service	Service-specific	Vehicle domain

Exact service URLs should be controlled through environment variables rather than hard-coded throughout the application.

3. Authentication API

POST /auth/register

Creates a new user.

Example request:

```
{
  "name": "Muhammad Bilal",
  "email": "user@example.com",
  "password": "secure-password"
}
```

Expected behavior:

- Validate input.
- Reject invalid email/password data.
- Reject an existing account where applicable.
- Hash the password.
- Create the user.
- Return appropriate account/authentication information.

POST /auth/login

Authenticates an existing user.

Example:

```
{
  "email": "user@example.com",
  "password": "secure-password"
}
```

Expected behavior:

- Validate credentials.
- Issue a JWT when authentication succeeds.
- Return user identity and role information required by the frontend.

4. Vehicle API

The vehicle service is responsible for vehicle domain operations.

POST /vehicles

Create a vehicle.

Authorization should be restricted to roles permitted to create vehicles.

GET /vehicles

List vehicles.

Supported query capabilities should include filtering/search where implemented.

GET /vehicles/:id

Retrieve a specific vehicle.

PATCH /vehicles/:id

Update an existing vehicle.

Only permitted fields should be accepted.

PATCH /vehicles/:id/status

Update the vehicle status.

Status values should be validated against the service’s controlled status set.

DELETE /vehicles/:id

Deactivate a vehicle.

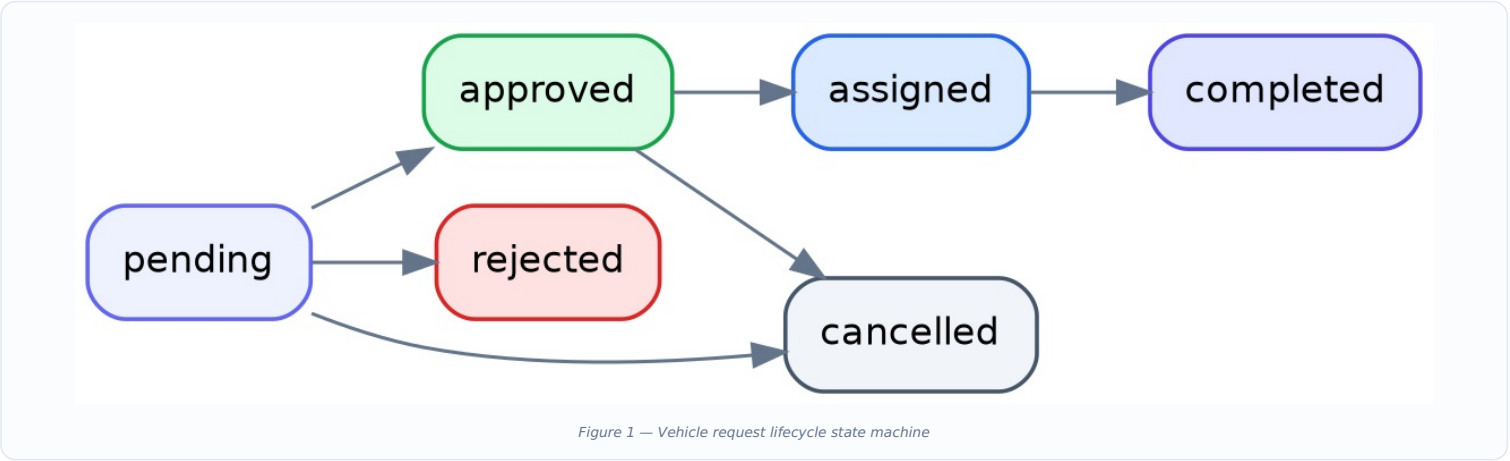
For fleet systems, deactivation is preferred over destructive deletion when historical records need to remain meaningful.

5. Vehicle Request API

Planned/expanding API:

```
POST /vehicle-requests
GET /vehicle-requests
GET /vehicle-requests/:id
PATCH /vehicle-requests/:id/status
```

Request lifecycle should use controlled states such as:



The exact state machine should be finalized before implementing advanced request transitions.

6. Vehicle Service/History API

Planned/expanding API:

```
POST /vehicles/:vehicleId/services
GET /vehicles/:vehicleId/services
GET /vehicle-services/:id
PATCH /vehicle-services/:id
```

A service record should contain enough information to explain what happened to a vehicle and when.

Typical fields:

```
{
  "vehicleId": "vehicle-id",
  "serviceDate": "2026-09-26",
  "serviceType": "Routine Service",
  "description": "Oil and filter replacement",
  "provider": "Service Center",
  "cost": 15000,
  "notes": "Next service due after 5000 km"
}
```

Field names should follow the actual implementation schema when the endpoint is finalized.

7. Error Response Convention

Recommended structure:

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email address"
    }
  ]
}
```

Production error responses should not expose stack traces, secrets, database connection information, or internal implementation details.

8. HTTP Status Codes

Code	Meaning
200	Successful request
201	Resource created
204	Successful request with no response body
400	Invalid request
401	Authentication required/invalid
403	Authenticated but unauthorized
404	Resource not found
409	Conflict
422	Validation failure where used
429	Rate limit exceeded
500	Unexpected server error

9. Authentication Header

Protected API requests should use:

```
Authorization: Bearer <JWT>
```

The exact token storage strategy should prioritize security and should be documented whenever it changes.

10. API Versioning

Versioning can be introduced when public contracts become stable:

```
/api/v1/vehicles
/api/v1/vehicle-requests
```

Until then, avoid unnecessary versioning complexity during rapid development.

11. API Documentation Rules

Whenever an endpoint changes:

1. Update the implementation.
2. Update validation.
3. Update authorization.
4. Update this document.
5. Update frontend API calls.
6. Test success and failure cases.
7. Add/update automated tests when the relevant test infrastructure exists.